by the closest of the defined amplitudes represented by the horizontal lines. In Figure 1.07, sample values 1 and 4 will be an accurate estimate of the actual amplitude because the wave is touching an amplitude line. In contrast, samples 5 and 6 will not be accurate because the actual amplitude is approximately half way between the two closest defined values.

To code sound, we need to make two decisions. The first is the number of bits we will use to store the amplitude values, which defines the **sampling resolution**. If we use only three bits then eight levels can be defined as shown in Figure 1.07. If too few are used there will be a significant error when the closest amplitude in the scale of values dictated by the sampling resolution is used as the approximation for the real value. In practice, 16 bits provides reasonable accuracy for most digitised sound.

We also need to choose the **sampling rate**, which is the number of samples taken per second. This should be in accordance with Nyquist's theorem which states that sampling must be done at a frequency at least twice the highest frequency of the sound in the sample.

KEY TERMS

Analogue data: data obtained by measurement of a physical property which can have any value from a continuous range of values

Digital data: data that has been stored as a binary value which can have one of a discrete range of values

Sampling: taking measurements at regular intervals and storing the value

Sampling resolution: the number of bits used to store each sample

Sampling rate: the number of samples taken per second

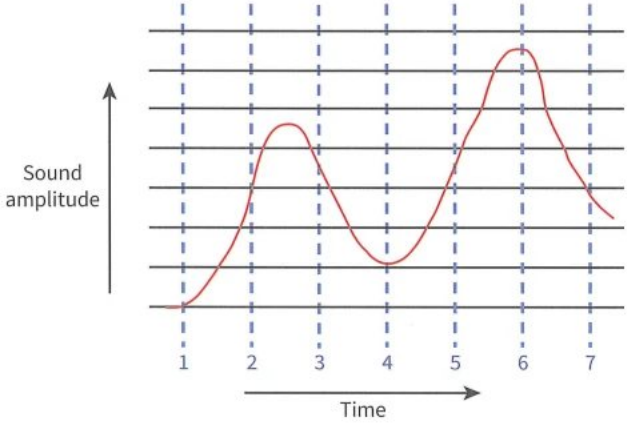


Figure 1.07 ADC sampling

Once again file size can be an issue. An increased sampling rate and an increased sampling resolution will both cause an increase in file size.

1.07 Compression techniques

Larger files require larger storage capacity but more importantly, larger files have lower transmission or download rates. For this reason, compression techniques are often used to reduce file size.

There are two categories of compression. The first is **lossless compression** where the file size is reduced but no information is lost. The process can be reversed to re-create the

original file. The second is **lossy compression** where the file size is reduced with some loss of information and the exact original file can never be recovered. In many applications a combination of lossless and lossy methods are used.

KEY TERMS

Lossless compression: coding techniques that allow subsequent decoding to recreate exactly the original file

Lossy compression: coding techniques that cause some information to be lost so that the exact original file cannot be recovered in subsequent decoding

We could use the same type of lossless file compression for everything, because all files contain binary codes. A good compression application will recognise patterns in files that it can compress, without any knowledge of what file type the code represents. However, most compression techniques have been developed to work with a particular type of file.

A common lossless compression technique is run-length encoding. This works particularly well with a bitmap file. The idea is that compression converts sequences of the same byte value into a code that defines the byte value and the number of times it is repeated (the count).

For example, the sequence of the same four bytes:

01100110 01100110 01100110 01100110

could be replaced by:

00000100 01100110

which says that there is a run of four of the bytes.

However, this is not the full story because in this simple form it is not obvious which byte represents the number (count) in the sequence. There are a number of methods used to distinguish the count byte from a data byte, but we do not need to go into the details.

If a file contains text, then compression must be lossless because any loss of information would lead to errors in the text. One possible compression method is called Huffman coding. The procedure used to carry out the compression is quite detailed, but the principle is straightforward. Instead of having each character coded in one byte, the text is analysed to find the most often used characters. These are then given shorter codes. The original stream of bytes becomes a bit stream.

A possible set of codes if a text contained only eight different letters is shown in Table 1.10. The important point to note here is the prefix property. None of the codes begins with the sequence of bits representing a shorter code. This means that there can be no ambiguity when the transmitted compressed file has to be converted back to the original text.

Code	Character
10	e
01	t
111	o
110	h
0001	l
0000	p
0011	w
0010	z

Table 1.10 An example of Huffman coding

Huffman coding can also be used for compressing a sound file. This is effective because some values for the amplitude occur far more often than others do.

If a vector graphic file needs to be compressed it is best converted to a Scalable Vector Graphics format. This uses a markup language description of the image which is suitable for lossless compression.

Lossy compression can be used in circumstances where a sound file or an image file can have some of the detailed coding removed or modified. This can happen when it is likely that the human ear or eye will hardly notice any difference. One method for lossy compression of a sound file takes advantage of the fact that the successive sampled values are unlikely to change very much. The file of individual sample amplitudes can be converted to a file of amplitude differences. Compression is achieved by using a lower sample resolution to store the differences. An alternative is to convert the sampled amplitudes that represent time domain data and transform them to a frequency domain representation. The values for frequencies that would be barely audible are then re-coded with fewer bits before the data is transformed back to the original time domain form.

For a bitmap a simple lossy compression technique is to establish a coding scheme with reduced colour depth. Then for each pixel in the original bitmap the code is changed to the one in the new scheme which represents the closest colour.

Extension Question 1.01

Graphic files can be stored in a number of formats. For example, JPEG, GIF, PNG and TIFF are just a few of the possibilities. What compression techniques, if any, do these use?

Reflection Point:

Can you recall the different possibilities for what one byte might be coded to represent?

Summary

- A binary code or a binary number can be documented as a hexadecimal number.
- Internal coding of signed integers is usually based on a two's complement representation.
- Binary addition can cause overflow.
- BCD is a convenient coding scheme for single denary digits.
- ASCII and Unicode are standardised coding schemes for text characters.
- An image can be stored either in a vector graphic file or in a bitmap file.
- An ADC works by sampling a continuous waveform.
- Lossless compression allows an original file to be recovered by a decoder; lossy compression irretrievably loses some information.